TASK 6 – Read and Print Data Correctly:

6.1 Format specifiers

Type / value	printf specifier	scanf specifier	Note
int	%d	%d	Use for ordinary signed integers.
unsigned int	%u	%u	Non-negative integer.
long	%ld	%ld	Specifier must match the argument type.
long long	%lld	%lld	Suitable for wide totals.
float	%f	%f	In printf, float is promoted to double; %f is used.
double	%f	%lf	For scanf, %lf reads a double.
char	%c	%c	Whitespace may need to be skipped with a leading space: " %c".
string	%s	%s	Use a width limit or prefer fgets for lines.

6.2 Validate scanf return values:

```
#include <stdio.h>

int main(void)
{
    int previous, current;
    printf("Enter previous and current readings: ");

    if (scanf("%d %d", &previous, &current) != 2) {
        fprintf(stderr, "Invalid input: enter two integers.\n");
        return 1;
    }
    if (previous < 0 || previous > 65535 ||
        current < 0 || current > 65535) {
        fprintf(stderr, "Readings must be between 0 and
65535.\n");
        return 1;
    }

    printf("Readings accepted: %d and %d\n", previous,
current);
    return 0;
}
```

.

scanf returns the number of successfully assigned input items. If the user enters letters instead of numbers, the return value will be less than the expected count and the program should reject the input. Using a wrong format specifier, such as %d for a long argument in printf, causes undefined behaviour; the output cannot be relied on

6.3 Print rupees and paise:

```
long long paise = 16250;
printf("Rs %lld.%02lld\n", paise / 100, paise % 100);
/* Output: Rs 162.50 */
```

6.4 fgets plus conversion alternative:

```
#include <stdio.h>
#include <stdlib.h>
#include <errno.h>
#include <limits.h>

int parse_reading(const char *line, long *value)
{
```

```
 char *end;
    errno = 0;
    long v = strtol(line, &end, 10);
    if (line == end || errno == ERANGE || v < 0 || v >
65535)
        return 0;
    while (*end == ' ' || *end == '\\t' || *end == '\\r' || *end
== '\\n')
        end++;
    if (*end != '\\0')
        return 0;
    *value = v;
    return 1;
}
```

fgets reads a whole line into a fixed-size buffer, after which strtol can validate conversion and range. Processing a line of length L is O(L), and buffer space is O(L) up to the chosen maximum line length. scanf can be concise, but line-based input is often easier to recover from because the whole invalid line can be discarded before the next attempt.